Alma Mater Studiorum-Università di Bologna
Scuola di Ingegneria

Enumerativi

Corso di Laurea in Ingegneria Informatica

Anno accademico 2025/2026

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA
IL PRESENTE MATERIALE È RISERVATO AL PERSONALE DELL'UNIVERSITÀ DI BOLOGNA E NON PUÒ ESSERE UTILIZZATO AI TERMINI DI LEGGE DA ALTRE PERSONE O PER FINI NON ISTITUZIONALI

ENUMERATIVI: L'EVOLUZIONE

- Il linguaggio C offre gli enumerativi (keyword **`enum`**), ma sono poco più che *alias per costanti intere*
  - compatibili con gli interi (e quindi mischiabili orrendamente..)
  - essendo "praticamente degli int", *non possono avere comportamento associato* → necessità di helper functions piazzate "da qualche parte"
- Java, Scala, Kotlin (in misura minore C#) offrono un enumerativo ben più completo, basato sul *typesafe enum pattern*
  - stessa keyword **`enum`**, ma sostanza molto diversa
  - un tipo enumerativo è una [quasi] normale classe, di cui però l'utente *non può costruire istanze a piacere*: il costruttore è privato
  - i valori elencati nell'enumerativo sono le *uniche possibili istanze* della classe, esposte come costanti (**`final`**)
  - in quanto classi, questi enumerativi possono avere *stato* e *metodi*

ENUMERATIVI Java

- Nella declinazione più semplice, per dichiarare un tipo enumerativo basta elencarne i valori:

```
public enum Direction {
    NORTH, SOUTH, EAST, WEST;
}
```

Java

- Questa dichiarazione causa la costruzione di *quattro istanze pubbliche e statiche* della classe **`Direction`**
- Si possono usare semplicemente con il loro nome assoluto:

```
Direction dir = Direction.NORTH;
```

  – fino a Java 20, dentro uno **`switch`** va usato *il nome relativo*
  – da Java 21, dentro uno **`switch`** si può usare anche *il nome assoluto*

ENUMERATIVI C#

- Anche in C# nel caso più semplice per dichiarare un tipo enumerativo basta elencare i valori, ma senza ';' finale:

```
public enum Direction {
    NORTH, SOUTH, EAST, WEST
}
```

C#

- Però qui le costanti si usano *sempre* con il *nome assoluto*:

```
Direction dir = Direction.NORTH;
```

  – si usa il nome assoluto anche *dentro* uno **`switch`**

ENUMERATIVI Scala

- Scala aveva in origine enumerativi più scomodi
- MA Scala 3 ha adottato lo stesso approccio di Java

Scala

```
enum Direction {
 case NORTH, SOUTH, EAST, WEST
}
```

– con la sintassi compatta, senza parentesi:

Scala 3

```
enum Direction : // quiet syntax
  case NORTH, SOUTH, EAST, WEST
```

- Anche qui le costanti si usano *sempre* con il *nome assoluto*:

```
Direction dir = Direction.NORTH;
```

– si usa il nome assoluto anche *dentro* il costrutto **`match`** (**`~switch`**)

ENUMERATIVI Kotlin

- In Kotlin la situazione è quasi identica a Java, ma la keyword **`enum`** è un «aggettivo» del «sostantivo» **`class`**:

```
public enum class Direction {
    NORTH, SOUTH, EAST, WEST;
}
```

Kotlin

- Come in C# e Scala, le costanti enumerative si usano sempre con il loro nome assoluto:

```
var dir : Direction = Direction.NORTH;
```

  – come in C#, esso va usato anche *dentro* uno **`switch`**
  – MA in Kotlin lo **`switch`** si chiama **`when`** e ha una sintassi innovativa (vedere oltre)

Java: UN SEMPLICE ESEMPIO

- Esempio di uso dentro uno **`switch`**:

Java

```
Direction dir = Direction.NORTH;
switch (dir) {
  case NORTH: System.out.println("North"); break;
  case SOUTH: System.out.println("South"); break;
  case EAST:  System.out.println("East");  break;
  case WEST:  System.out.println("West");  break;
}
```

Da Java 21, anche in forma qualificata **`Direction.NORTH`**, etc.

- Però, questo approccio non è scalabile: lo **`switch`** diventa rapidamente illeggibile, non manutenibile, ridondante
- Molto meglio *sfruttare il fatto che l'enum sia una classe* per *personalizzarla* e introdurre *metodi appositi.*

LO STESSO ESEMPIO IN C#

- Esempio di uso dentro uno **`switch`** in C#:

C#

```
Direction dir = Direction.NORTH;
switch (dir) {
  case Direction.NORTH: Console.WriteLine("North"); break;
  case Direction.SOUTH: Console.WriteLine("South"); break;
  case Direction.EAST:  Console.WriteLine("East");  break;
  case Direction.WEST:  Console.WriteLine("West");  break;
}
```

L'ESEMPIO IN Scala…

- In Scala, l'antico (e obsoleto..) costrutto **`switch`** del C viene sostituito da un più moderno e potente costrutto **`match`**

Scala

```
dir match {
  case Direction.NORTH => println("North");
  case Direction.SOUTH => println("South");
  case Direction.EAST  => println("East");
  case Direction.WEST  => println("West");
}
```

- Cambia la sintassi, perché cambia la semantica:
  - i rami diventano mutuamente esclusivi → *non occorre più il* ***`break`*** (la cui dimenticanza ha causato negli anni tanti bug…)
  - la struttura «x **`match`** expression» evidenzia l'idea che x possa fare match con tante situazioni, *anche ben oltre le semplici costanti*

…e in Kotlin

- Anche in Kotlin l'antico e obsoleto costrutto **`switch`** viene sostituito da un costrutto più moderno e potente: **`when`**

Kotlin

```
when(dir) {
  Direction.NORTH -> println("North");
  Direction.SOUTH -> println("South");
  Direction.EAST  -> println("East");
  Direction.WEST  -> println("West");
}
```

  – come in Scala, i rami sono mutuamente esclusivi → niente **`break`**
  – estetica: cambia la freccina (da «cicciotta» **=>** a «snella» **->**)
  – sparisce le keyword **`case`**, ritenuta inutilmente prolissa
  – la struttura è una via di mezzo fra Java (**`when`** iniziale) e Scala (il costrutto fa match con le più varie situazioni, *ben oltre le costanti)*

Dall'antico `switch` a `match/when`

- I costrutti **`match`** (Scala) e **`when`** (Kotlin) possono essere usati anche come espressioni che restituiscono un valore

Scala

```
val name = dir match {
  case Direction.NORTH => "North";
  case Direction.SOUTH => "South";
  case Direction.EAST  => "East";
  case Direction.WEST  => "West";
}
```

Mutuamente esclusive, senza **`break`**

Kotlin

```
val name = when(dir) {
  Direction.NORTH -> "North";
  Direction.SOUTH -> "South";
  Direction.EAST  -> "East";
  Direction.WEST  -> "West";
}
```

Mutuamente esclusive, senza **`break`**

`switch` expression in Java 16+

- Questo approccio ha avuto tale successo che da Java 16 è previsto anche in Java come *switch expression*

```
var name = switch(dir) {
  case Direction.NORTH -> "North";
  case Direction.SOUTH -> "South";
  case Direction.EAST  -> "East";
  case Direction.WEST  -> "West";
};
```

Java

Mutuamente esclusive, senza **`break`**

Da Java 21, anche in forma qualificata **`Direction.NORTH`**, etc.

  - a differenza dello **`switch`** classico (che è un'istruzione), la **`switch`** expression *restituisce un valore*
  - si riconosce per la presenza del separatore **`->`** anziché **`:`**
  - nella switch expression i rami diventano mutuamente esclusivi → non occorre più il **`break`**
  - MA i rami devono coprire tutti i casi (eventualmente con **`default`**)

Java: ENUM COME CLASSE

- L'**`enum`** Java è in realtà una *classe gestita dal compilatore in modo speciale*, aggiungendo automaticamente:
  - un costruttore privato
  - un metodo di utilità **`ordinal`**
    - restituisce *l'indice di quel valore* nell'elenco degli enumerativi (ossia 0 per `NORTH`, 1 per `SOUTH`, etc.)
  - due metodi statici *factory*, ***`values`*** e ***`valueOf`***
    - **`values()`** restituisce *un array* con *tutti i possibili valori* dell'enumerativo
    - **`valueOf(String)`** restituisce *l'oggetto corrispondente* alla stringa passata (ossia da `"NORTH"` restituisce `Direction.NORTH`, etc.)

Java: ENUM COME CLASSE

- Quindi la classe è compilata circa così:

Java

```
public final class Direction {
  public static final Direction NORTH = new Direction();
  public static final Direction SOUTH = new Direction();
  public static final Direction EAST  = new Direction();
  public static final Direction WEST  = new Direction();
  private Direction() {...} // costruttore privato
  public static Direction[] values() {
        return un array con tutti i possibili valori dell'enumerativo
  }
  public static Direction valueOf(String s) {
        return l'opportuna istanza dell'enumerativo
  }
  public int ordinal() {
        return l'indice corrispondente all'enumerativo corrente (this)
  }
}
```

ENUM IN Java: ESEMPIO D'USO

- Questo semplice frammento mostra come usarli
  - `Direction.values()` fornisce *i valori su cui iterare*
    - poi, per ogni iterazione si effettua l'operazione (es. si stampa l'indice)
  - `Direction.valueOf()` converte una stringa nel corrispondente oggetto dell'enumerativo

```
for (Direction d : Direction.values())
    System.out.println( d.ordinal() );
System.out.println(Direction.valueOf("EAST"));
```

Java

Output

```
0
1
2
3
EAST
```

ENUM IN C#: ESEMPIO D'USO

- Nel caso C#, i metodi disponibili sono diversi e *meno immediati* nell'uso (richiedono **typeof** e a volte cast):
  - **GetName** recupera il nome dell'i-esima costante dell'enumerativo
  - **GetNames** restituisce un array con tutti i *nomi* delle costanti
  - **GetValues** restituisce un array con tutti i *valori* delle costanti
  - **Parse** recupera la costante dato il suo nome

Necessario perché i metodi valgono per tutti gli enum..

```
Console.WriteLine(
            Direction.GetName(typeof(Direction),1) );

string[] allNames = Direction.GetNames(typeof(Direction));
foreach(string s in allNames) Console.WriteLine(s);
foreach(int v in Direction.GetValues(typeof(Direction)) …
Direction shouldBeSouth =
  (Direction) Direction.Parse(typeof(Direction), "SOUTH");
Console.WriteLine(shouldBeSouth);
```

Cast necessario

C#

L'ESEMPIO IN Scala

- In Scala la situazione è praticamente identica a Java
  - metodi **`values`**, **`valueOf`**, **`ordinal`**
    - NB: **`ordinal`** non accetta argomenti, quindi va invocato senza **`()`**
  - metodo aggiuntivo **`fromOrdinal`** per ottenere l'i-esimo valore

ESEMPIO in sintassi classica:

```
for (d <- Direction.values )
      println( d.ordinal );
println(Direction.valueOf("EAST"));
```

Scala

```
0
1
2
3
EAST
```

```
for (d <- Direction.values )
      println( d.ordinal );
println(Direction.fromOrdinal(2));
```

Scala

```
0
1
2
3
EAST
```

L'ESEMPIO IN Scala 3

- Stessi esempi con la sintassi compatta
  - al costrutto **for** è aggiunta la keyword **do**
  - non sono più richieste le parentesi attorno al generatore

ESEMPIO in sintassi compatta Scala 3:

```
for d <- Direction.values do
      println( d.ordinal );
println(Direction.valueOf("EAST"));
```

Scala 3

```
0
1
2
3
EAST
```

```
for d <- Direction.values do
      println( d.ordinal );
println(Direction.fromOrdinal(2));
```

Scala 3

```
0
1
2
3
EAST
```

L'ESEMPIO IN Kotlin

- Anche in Kotlin la situazione è pressoché identica a Java
  - unica differenza: **`ordinal`** non è più un metodo, ma un valore intero (read-only) associato alla specifica istanza
  - l'effetto netto è che, come in Scala, si usa *senza parentesi*

Kotlin

```
for (d in Direction.values())
      println( d.ordinal );
println(Direction.valueOf("EAST"));
```

```
0
1
2
3
EAST
```

ESTENDERE L'ENUMERATIVO

- Essendo l'enum una [quasi] normale classe, nulla vieta che *aggiungiamo noi altri metodi,* ad esempio per:
  - **`getOpposite`**: restituire la direzione opposta a quella data
  - …

Non C# !

Java

```
public enum Direction {
    NORTH, SOUTH, EAST, WEST;
    public Direction getOpposite(){
        return switch (this) {
            case NORTH -> SOUTH;
            case SOUTH -> NORTH;
            case EAST  -> WEST;
            case WEST  -> EAST;
        };
    }
}
```

`getOpposite` è un normale metodo, quindi **`this`** è la specifica istanza dell'enumerativo su cui il metodo è chiamato.

ESEMPIO D'USO:

```
IO.println(
    dir.getOpposite());
```

Questo non si può fare in C#: il costrutto **`enum`** NON è estendibile come una normale classe

ESTENDERE L'ENUMERATIVO

- Essendo l'enum una [quasi] normale classe, nulla vieta che *aggiungiamo noi altri metodi,* ad esempio per:
  - **`getOpposite`**: restituire la direzione opposta a quella data
  - **`getDegrees`**: restituire i gradi (come nella bussola) corrispondenti

Java

```
public enum Direction {
    NORTH, SOUTH, EAST, WEST;
    ...
    public int getDegrees(){
        return switch (this) {
            case NORTH ->   0;
            case SOUTH -> 180;
            case EAST  ->  90;
            case WEST  -> 270;
        };
    }
}
```

Anche **`getDegrees`** è un normale metodo

ESTENDERE L'ENUMERATIVO

- Essendo l'enum una [quasi] normale classe, nulla vieta che *aggiungiamo noi altri metodi,* e naturalmente anche
  - **`toString`**: restituisce una frase che rappresenti il contenuto

Java

```
public enum Direction {
    NORTH, SOUTH, EAST, WEST;
    ...
    public String toString(){
        return switch (this) {
            case NORTH -> "Nord";
            case SOUTH -> "Sud";
            case EAST  -> "Est";
            case WEST  -> "Ovest";
        } + " a " + getDegrees() + "°";
    }
}
```

…e anche **`toString`**, dopo tutto!

UN POSSIBILE MAIN

- Piccolo main di prova:

```
class Main  {
  public static void main(String[] args) {
    Direction dir = Direction.NORTH;
    IO.println("Direzione attuale: " + dir);
    IO.println("Direzione opposta: " + dir.getOpposite());
  }
}
```

Java

RICORDA: la chiamata a **`println`** invoca indirettamente **`toString`** per generare la frase da stampare.

- Risultato atteso:

```
Direzione attuale: Nord a 0°
Direzione opposta: Sud a 180°
```

PROBLEMA: LO SWITCH-one!

- PROBLEMA: il codice precedente evidenzia ***l'inaccettabile replicazione di codice*** e *l'approccio estremamente verboso e ripetitivo,* perché ***ogni metodo contiene uno switch con tutti i possibili valori delle costanti!*** ☹
- Un tale «terribile switch-one» è inaccettabile perché:
  - è impraticabile se le costanti sono molte
    - pensate alle 52 carte da gioco…
  - dà luogo a codice fragilissimo, che «si rompe» con niente
    - cosa succede se l'utente aggiunge una costante enumerativa?
    - con lo switch statement, gli switch falliscono pericolosamente
    - con la switch expression, che richiede di coprire tutti i casi, compaiono imprevisti errori di compilazione che non c'erano

COME RISOLVERE?

- IDEA: visto che un enumerativo è di fatto una classe, *può avere propri dati,* oltre che metodi
- Si può quindi pensare di *inserire in ogni istanza-costante i dati "corrispondenti"* ☺, associando già al momento delle definizione delle costanti:
  - all'istanza NORTH la stringa "Nord" e 0°
  - all'istanza SOUTH la stringa "Sud" e 180°
  - … e così via
- Per farlo, occorre *definire esplicitamente il costruttore (privato!) con i relativi argomenti*

ENUM COME CLASSE Java PERSONALIZZATA

- Oltre ad aggiungere metodi, possiamo *aggiungere proprietà* e farle *inizializzare normalmente* dal costruttore (privato)
- Ne definiamo due:
  - **value**: la stringa "corrispondente"
  - **degrees**: i "gradi" corrispondenti (0°=nord, 90°=est, etc.)

Java

```
private String value;
private int degrees;

private Direction(String value, int degrees) {
        this.value = value;
        this.degrees = degrees;
}
```

- La dichiarazione dell'enumerativo va adeguata di conseguenza, passando a ogni costante le due proprietà corrispondenti.

ENUM COME CLASSE Java PERSONALIZZATA

- Versione revisionata:

Java

```
public enum Direction {
   NORTH("Nord", 0), SOUTH("Sud", 180),
   EAST("Est", 90),  WEST("Ovest", 270);
   private String value;
   private int degrees;
   private Direction(String value, int degrees) {
      this.value = value; this.degrees = degrees;
   }
   public String toString(){
      return value + " a " + degrees + "°";
   }
   ... // SEGUE
```

ENUM COME CLASSE Java PERSONALIZZATA

- Versione revisionata:

Java

```
public enum Direction {
   // ... CONTINUA
   public int getDegrees(){
      return degrees;
   }
   public Direction getOpposite() {
      return switch (this) {
        case NORTH -> SOUTH;
        case SOUTH -> NORTH;
        case EAST  -> WEST;
        case WEST  -> EAST;
      };
    }
}
```

IL SOLITO MAIN

- Solito main di prova *(non richiede neppure ricompilazione!)*:

Java

```
class Main  {
  public static void main(String[] args) {
    Direction dir = Direction.NORTH;
    IO.println("Direzione attuale: " + dir);
    IO.println("Direzione opposta: " + dir.getOpposite());
  }
}
```

RICORDA: la chiamata a **`println`** invoca indirettamente **`toString`** per generare la frase da stampare.

- Risultato atteso:

```
Direzione attuale: Nord a 0°
Direzione opposta: Sud a 180°
```

UN PRIMO BILANCIO ☺

- **Arricchendo la rappresentazione dell'enumerativo** abbiamo migliorato nettamente le cose, perché abbiamo *trasferito i dati relativi alle varie costanti lì dove ha senso che stiano: nella costante corrispondente!*
- **Lesson Learned: PIÙ DATI = MENO CODICE**
  - Arricchire la rappresentazione dell'oggetto non solo è più pulito: fa risparmiare molto codice ripetitivo dopo, perché evita di dover inserire nei metodi (replicandola) la logica necessaria a "dedurre" con quale oggetto si abbia a che fare per calcolarne una caratteristica
  - **Ricordiamolo sempre!**

UN PRIMO BILANCIO ☺

- Ciò ha consentito di *evitare completamente gli switch* nei metodi toString e getDegrees:
  - tali metodi dovevano infatti faticare per *sintetizzare dati che non avevano,* chiedendosi ogni volta "chi fosse" l'oggetto corrente con una serie di test;
  - ora invece possono semplicemente chiedere all'oggetto corrente *l'informazione che esso stesso possiede!* ☺
- Però, non l'abbiamo fatto per getOpposite.. Come mai? 😐
  - 1° MOTIVO: una ragione di principio
    Ha senso inglobare nelle costanti *caratteristiche «naturali»* dell'entità, *non il risultato di calcoli o elaborazioni* (la valutazione è lasciata alla sensibilità ed esperienza del progettista, visto il contesto applicativo)
  - 2° MOTIVO: una ragione tecnica
    La soluzione più «ovvia» porta a un tipico caso di *forward reference*

UN PASSO IN PIÙ…? (o no?)

- Poiché l'appetito vien mangiando, potrebbe venire naturale *pensare di aggiungere alla definizione di ogni costante anche la sua «opposta»,* così da applicare lo stesso approccio anche a getOpposite:

```
public enum Direction {

   NORTH("Nord", 0, SOUTH),SOUTH("Sud", 180, NORTH),
   EAST("Est", 90, WEST),  WEST("Ovest", 270, EAST);
   ...
}
```

Java

- **MA questo è impossibile**, perché *non si può usare una costante in una definizione prima di averla definita!*
- È un caso di *forward reference*, vietata (non solo in Java!)

UN PASSO IN PIÙ…? (o no?) COME RISOLVERE?

- Una vera soluzione «davvero pulita» non c'è
- Fra i possibili workaround, uno dei meno invasivi può essere *passare il nome della costante* anziché la costante stessa
  - si potrà poi **sintetizzare da quello** la corrispondente costante enumerativa in *getOpposite*, **tramite il metodo di utilità valueOf** disponibile in tutti gli enumerativi

Java

```
public enum Direction {
   NORTH("Nord", 0, "SOUTH"), SOUTH("Sud", 180, "NORTH"),
   EAST("Est", 90, "WEST"),   WEST("Ovest", 270, "EAST");
   // qui va il costruttore a tre argomenti
   public Direction getOpposite() {
         return Direction.valueOf(oppositeName);
   }
}
```

Con questo approccio è CRUCIALE L' IMPORTANZA DEL TEST perché se un nome è sbagliato… BOOM!

ENUM COME CLASSE Scala PERSONALIZZATA

- Lo stesso esempio in Scala (sintassi classica):

Scala

```
enum Direction(value: String, degrees:Int) {
  case NORTH extends Direction("Nord", 0);
  case SOUTH extends Direction("Sud", 180);
  case EAST  extends Direction("Est", 90);
  case WEST  extends Direction("Ovest", 270);
  override
  def toString() =  value + " a " + degrees + "°";
  def getOpposite() = this match {
      case NORTH => SOUTH;
      case SOUTH => NORTH;
      case EAST =>  WEST;
      case WEST =>  EAST;
   }
}
```

```
Nord a 0°
Sud a 180°
```

ENUM COME CLASSE Kotlin PERSONALIZZATA

- Lo stesso esempio in Kotlin

```
enum class Direction(val value: String, val degrees:Int) {
  NORTH("Nord", 0),
  SOUTH("Sud", 180),
  EAST("Est", 90),
  WEST("Ovest", 270);
  override fun toString() = value + " a " + degrees + "°";
  fun getOpposite() = when(this) {
      NORTH -> SOUTH;
      SOUTH -> NORTH;
      EAST ->  WEST;
      WEST ->  EAST;
   }
}
```

`val` per poterli usare in toString

Kotlin

Analoghi a Java

```
Nord a 0°
Sud a 180°
```

PERSONALIZZARE GLI ENUM IN C#

- In C# la personalizzazione degli enum è più limitata
- È possibile però *associare un valore* alle costanti all'atto stesso della loro dichiarazione, tramite l'operatore =
  - ad esempio, i "gradi" corrispondenti (0°=nord, 90°=est, etc.)

```
public enum Direction {

    NORTH=0, SOUTH=180, EAST=90, WEST=270

}
```

Non in Java

C#

- Il metodo `GetValues` in questo caso restituirebbe 0, 180, 90, 270 anziché i classici 0, 1, 2, 3
- Il metodo `format`, che produce *la stringa corrispondente* alla costante enum, può essere *istruito per restituire il valore associato* anziché il nome della costante stessa

ENUM C#: ESEMPIO D'USO

- Stampiamo la direzione EAST in diversi formati:

C#

```
string value = Direction.Format(
        typeof(Direction), Direction.EAST, fmt);
Console.WriteLine(value);
```

  – il formato `fmt` può essere "G", "F", "X", "D" (anche minuscole)
  – `"G"` produce il nome della costante (es. "EAST")

EAST

  – `"F"` produce il nome della costante senza tenere conto di flag
  – `"D"` produce il nome della costante (se non ci sono valori numerici specificati) o il suo valore numerico decimale (se specificato)

90

  – `"X"` è identica a `"D"` ma stampa il valore in esadecimale

0000005A

UN ALTRO ESEMPIO: DayOfWeek in Java

- **`DayOfWeek`** è un enumerativo del package **`java.time`** che rappresenta i sette giorni della settimana.
- Stampiamo tutti i valori e i rispettivi indici (0-based):

Java

```
for (DayOfWeek day : DayOfWeek.values())
    System.out.println(day.ordinal() + ": " + day);
```

```
0: MONDAY
1: TUESDAY
2: WEDNESDAY
3: THURSDAY
4: FRIDAY
5: SATURDAY
6: SUNDAY
```

- Ovviamente, un indice 0-based è poco pratico
  - ha senso come rappresentazione interna, non come vista esterna!

UN ALTRO ESEMPIO: DayOfWeek in Java

- Per questo, **`DayOfWeek`** *integra i metodi standard degli enumerativi aggiungendone altri specifici e personalizzati*
- In particolare, **`getValue`** restituisce l'indice 1-based destinato alla vista esterna:

```
for (DayOfWeek day : DayOfWeek.values())

    System.out.println(day.getValue() + ": " + day);
```

Java

- È fornito anche il corrispondente metodo factory di conversione "da indice a oggetto", chiamato **`of`**:

```
DayOfWeek day = DayOfWeek.of(2); // martedì
```

NB: **`DayOfWeek`** e tutta la libreria **`java.time`** sono usati anche da Scala e Kotlin senza modifiche

Kotlin Scala

… e DayOfWeek in C#

- Ovviamente, anche C# offre la sua versione di **`DayOfWeek`**
  - ma qui i giorni sono numerati a partire dalla domenica!!
  - *ottimo motivo per usare SEMPRE le COSTANTI, mai i numeri!*

Java

```
0: MONDAY
1: TUESDAY
2: WEDNESDAY
3: THURSDAY
4: FRIDAY
5: SATURDAY
6: SUNDAY
```

C#

```
0: Sunday
1: Monday
2: Tuesday
3: Wednesday
4: Thursday
5: Friday
6: Saturday
```

```
foreach(int v in DayOfWeek.GetValues(typeof(DayOfWeek)))
 Console.WriteLine(v + ": " +
    DayOfWeek.GetName(typeof(DayOfWeek), v));
```

C#

Un caso di studio
Banconote e Portafogli

CASO DI STUDIO (1/15)
Banconote e monete

- Vogliamo definire i *tagli* possibili di *banconote* e *monete*
- La prima idea potrebbe essere banalmente questa:

Java

```
public enum Taglio  {
   CINQUECENTO, DUECENTO, CENTO, CINQUANTA,
   VENTI, DIECI, CINQUE, DUE, UNO;
}
```

- Peccato che così facendo diventi un incubo *ricavare il valore dal nome,* quando serve per fare calcoli!
  - ESEMPIO: sommare tutte le banconote e/o monete contenute nel vostro portafoglio
  - Cosa usiamo, uno **`switch`**?!?

CASO DI STUDIO (2/15)
Banconote e monete

- Con un modello così semplicistico, per estrarre il valore del generico taglio sarebbe inevitabile un orrendo **`switch`**-one:

Java

```
static int valore(Taglio t){
    return switch(t){
        case CINQUECENTO -> 500;
        case DUECENTO -> 200;
        case CENTO -> 100;
        case CINQUANTA -> 50;
        case VENTI -> 20;
        case DIECI -> 10;
        case CINQUE -> 5;
        case DUE -> 2;
        case UNO -> 1;
    };
}
```

È codice cablato, verboso e *fragile!*

Se l'enumerativo cambia (si aggiunge o toglie un valore), il codice *va modificato*

NB: rispetto allo switch classico, la switch expression almeno fa la verifica di copertura di tutti i casi

Se poi volessimo anche stampare il nome delle costanti in modo personalizzato, ce ne vorrebbe un altro!!

CASO DI STUDIO (3/15)
Refactoring

- Ma non ha senso: è proprio sbagliato il modello di base
  - il valore *è una proprietà della banconota o della moneta:* nel mondo reale, è scritto sopra!
  - non è una terza parte (una funzione di **`switch`**) a dover sapere quanto vale un taglio: *è il taglio che deve saperlo!*

- Refactoring

```
public enum Taglio  {
   CINQUECENTO(500), DUECEN
   CINQUANTA(50), VENTI(2
   DUE(2), UNO(1);
   private Taglio(int valore){ this.valore = valore; }
   private int valore;
   public int getValore() { return valore; }
}
```

Java

Codice robusto

Se l'enumerativo cambia (si aggiunge o si toglie un valore), *tutto continua a funzionare senza alcun intervento* ☺

Giustamente, così è il **`Taglio`** a sapere "i fatti del **`Taglio`**"!

CASO DI STUDIO (4/15)
Il portafoglio

- Nel mondo reale, banconote e monete si tengono in un «contenitore» detto *portafoglio*
- Essendo un contenitore, potremmo pensare di modellarlo ad esempio con un *array:*

```
Taglio[] portafoglio = new Taglio[] {
   Taglio.CINQUANTA, Taglio.VENTI, Taglio.DIECI,
   Taglio.DUE, Taglio.DUE, Taglio.UNO
};
```

Java

- Due domande:
  - Come calcolarne il valore?
  - Come stamparne il contenuto?

Se **`Taglio`** è ben modellato, è facile!

CASO DI STUDIO (5/15)
Calcolo del valore di un portafoglio

- Per calcolare il valore del portafoglio occorre *scorrere l'array,* sommando in una variabile i valori dei tagli:

```
static int valore(Taglio[] portafoglio){
   int sum = 0;
   for (Taglio t : portafoglio ) sum += t.getValore();
   return sum;
}
```

Java

RIFLETTI: sono funzioni statiche *perché non esiste davvero* una entità portafoglio

- Analogamente si procede per stamparne il contenuto:

```
static String contenuto(Taglio[] portafoglio){
   StringBuilder sb = new StringBuilder();
   for (Taglio t : portafoglio) sb.append(t + ", ");
   return sb.toString();
}
```

Java

CASO DI STUDIO (6/15)
Mini-collaudo

- Un piccolo main di prova

```
public static void main(String[] args)   {
 Taglio[] portafoglio = new Taglio[] {
        Taglio.CINQUANTA,
        Taglio.VENTI,
        Taglio.DIECI,
        Taglio.DUE, Taglio.DUE,
        Taglio.UNO
 };
 System.out.println("Contenuto portafoglio: " +
                                contenuto(portafoglio));
 System.out.println("Valore portafoglio: " +
                                valore(portafoglio));
}
```

Java

OSSERVA: le funzioni statiche precedenti vanno messe o nella stessa classe del main o in una qualche libreria accessoria: *non hanno una vera «casa»*

```
Contenuto portafoglio: CINQUANTA, VENTI, DIECI, DUE, DUE, UNO,
Valore portafoglio: 85
```

CASO DI STUDIO (7/15)
Portafoglio: situazione

- Fin qui, il portafoglio è «solo» un array di **`Taglio`**
- Per operare su esso abbiamo predisposto due funzioni statiche, «parcheggiate» nel cliente (main) o in una libreria

```
static int valore(Taglio[] portafoglio)
static String contenuto(Taglio[] portafoglio)
```

→ insoddisfacente, inelegante, inopportuno

- Ci sono altre opzioni?
  - predisporre una libreria ad hoc («WalletLibrary»)
  - collocarle altrove, in un luogo «adatto» e «attinente» (attinente…?)
  - OPPURE…. ☺

CASO DI STUDIO (8/15)
Le funzioni del portafoglio

- Opzione #1: `WalletLibrary`
  - sarebbe l'analogo di `FrazLib` per le frazioni
  - soluzione non inguardabile, ma neanche elegante: perché un altro componente software? Ce n'è davvero bisogno?

Java

```
public class WalletLibrary {
 static int valore(Taglio[] portafoglio){
    int sum = 0;
    for (Taglio t : portafoglio ) sum += t.getValore();
    return sum;
 }
 static String contenuto(Taglio[] portafoglio){
    StringBuilder sb = ne
    for (Taglio t : porta
    return sb.toString();
 }
}
```

```
public static void main(String[] args) {
 Taglio[] portafoglio = new Taglio[] {…};
 System.out.println("Contenuto portafoglio: "
        + WalletLibrary.contenuto(portafoglio));
 System.out.println("Valore portafoglio: "
        + WalletLibrary.valore(portafoglio));
}
```

CASO DI STUDIO (9/15)
Le funzioni del portafoglio

- Opzione #2: Per evitare l'introduzione di una libreria, si può pensare, dato che le due funzioni lavorano su una pluralità (array) di **`Taglio`**, di collocarle *per attinenza concettuale (…?)*, nella classe **`Taglio`**
  - MA la classe Taglio rappresenta UNA banconota o moneta: cosa c'entrano «davvero» delle funzioni che operano su *insiemi di tagli?*
  - *un rimedio peggiore del male: a chi verrebbe in mente di cercare in una banconota le funzioni che operano su un portafoglio???*

Java

```
public static void main(String[] args) {
 Taglio[] portafoglio = new Taglio[] {…};
 System.out.println("Contenuto portafoglio: "
        + Taglio.contenuto(portafoglio));
 System.out.println("Valore portafoglio: "
        + Taglio.valore(portafoglio));
}
```

CASO DI STUDIO (10/15)
Taglio + funzioni statiche ☹

Java

```
public enum Taglio  {
   CINQUECENTO(500), DUECENTO(200), CENTO(100),
   CINQUANTA(50), VENTI(20), DIECI(10), CINQUE(5),
   DUE(2), UNO(1);
   private Taglio(int valore){ this.valore = va
   private int valore;
   public int getValore() { return valore; }
   public static String contenuto(Taglio[] portafoglio){
        StringBuilder sb = new StringBuilder();
        for (Taglio t : portafoglio) sb.append(t + ", ");
        return sb.toString();
   }
   public static int valore(Taglio[] portafoglio){
        int sum = 0;
        for (Taglio t : portafoglio ) sum += t.getValore();
        return sum;
   }
}
```

Messe qui per *vicinanza concettuale (?)* alla nozione di **`Taglio`**

MA non è «davvero» casa loro!
Inquinano la pulizia della classe **`Taglio`**

CASO DI STUDIO (11/15)
Portafoglio come first-class entity

- Per fare le cose bene bisogna riconoscere che *c'è un concetto che sta lottando per venir fuori:* il *Portafoglio*
  - INFATTI, è un'entità che esiste nel mondo reale: ce l'avete in tasca o nella borsa!

- Serve un'entità **`Portafoglio`** come *classe* dotata di *suoi metodi*
  - costruttore a partire da un insieme (array) di **`Taglio`** già fissato
  - costruttore vuoto (+ metodo **`add`** per aggiungere banconote via via)
  - metodi **`valore`** e **`contenuto`** (magari rinominati **`toString`** !)
  - magicamente, tutto va a posto: *segno inequivocabile* che la soluzione è robusta, coerente e risolve *veramente* il problema *come va fatto*
- **`Taglio`** torna a essere quella di prima
  - senza funzioni statiche accessorie «messe lì» a inquinarla!

CASO DI STUDIO (12/15)
La classe Portafoglio

Java

```
public class Portafoglio {
   private Taglio[] contenuto;
   int logicalSize;
   public Portafoglio(int n){
     contenuto = new Taglio[n];
     int logicalSize = 0;
   }
   public void add(Taglio t){
     contenuto[logicalSize++] = t;
   }
   public Portafoglio(Taglio[] contenuto){
     this.contenuto = Arrays.copyOf(contenuto, contenuto.length);
     logicalSize = contenuto.length;
   }

   ... segue ...
```

`Portafoglio` incapsula l'array di **`Taglio`**

Ora finalmente il portafoglio esiste in quanto tale: non è solo una variabile con un nome «evocativo» definita «da qualche parte»!

Ha opportuni costruttori e metodi: *espone l'interfaccia esterna ritenuta più idonea*, non quella imposta dal fatto di essere un array!

CASO DI STUDIO (13/15)
La classe Portafoglio

```
public class Portafoglio {

   ... continua ...

   public int valore(){
     int sum = 0;
     for (int i=0; i<logicalSize; i++) sum += contenuto[i].getValore();
     return sum;
   }

   public String toString(){
     StringJoiner sj = new StringJoiner(", ");
     for (int i=0; i<logicalSize; i++) sj.add(portafoglio[i].toString());
     return sj.toString();
   }

}
```

Java

Ex metodi statici
Ora finalmente hanno trovato casa!

Già che ci siamo: invece di **StringBuilder, StringJoiner!**

```
Contenuto portafoglio: CINQUANTA, VENTI, DIECI, DUE, DUE, UNO
Valore portafoglio: 85
```

CASO DI STUDIO (14/15)
Portafoglio: collaudo

Java

```
public static void main(String[] args) {

  Portafoglio pf1 = new Portafoglio(10);
  for(Taglio t: new Taglio[] {
     Taglio.CINQUANTA, Taglio.VENTI, Taglio.DIECI, Taglio.DUE,
     Taglio.DUE, Taglio.UNO }) pf1.add(t);

  Portafoglio pf2 = new Portafoglio(
     new Taglio[] {
        Taglio.CINQUANTA, Taglio.CINQUANTA, Taglio.VENTI,
        Taglio.DIECI, Taglio.DUE, Taglio.DUE, Taglio.DUE, Taglio.UNO
     });
     System.out.println("Contenuto portafoglio: " + pf1);
     System.out.println("Valore portafoglio: "    + pf1.getValore());
     System.out.println("Contenuto portafoglio: " + pf2);
     System.out.println("Valore portafoglio: "    + pf2.getValore());
}
```

Costruisce un portafoglio inizialmente vuoto (poi lo riempie)

Costruisce un portafoglio già inizializzato con quel contenuto

```
Contenuto portafoglio: CINQUANTA, VENTI, DIECI, DUE, DUE, UNO
Valore portafoglio: 85
Contenuto portafoglio: CINQUANTA, CINQUANTA, VENTI, DIECI, DUE, DUE, DUE, UNO
Valore portafoglio: 137
```

CASO DI STUDIO (15/15)
Bilancio finale

- L'enumerativo basico *non manteneva in sé tutta la conoscenza* che lo riguardava
  - costringeva altri (i clienti) e fare i salti mortali per scrivere algoritmica
  - non si sapeva *dove mettere* tali algoritmi, perché "non avevano casa"
- L'enumerativo evoluto *incapsula tale conoscenza*
  - evita alla radice gli **`switch`** che *cercavano di ricostruire ex post ciò che mancava ex ante*
  - codice *robusto* ed *estendibile* al posto di codice fragile e ad hoc
  - ma per esprimersi al meglio va completato col concetto di *portafoglio*
- Il portafoglio incapsula la nozione "collezione di banconote"
  - L'idea di portafoglio *assurge a vera classe:* non è più solo un «bel nome evocativo» per un array piazzato «da qualche parte»
  - niente più funzioni statiche accessorie, tutto va al suo posto ☺